Railway level crossing in mCRL2

Diogo and Rodrigo Gomes de Araújo
up202510267, up202210267

November 2026

Introduction

This report was written for the first assignment of the *Systems Verification* course at *Faculdade de Ciências da Universidade do Porto* (FCUP). It presents a model of a railway level crossing, describing the interaction between a controller, a gate, and one or more trains that wish to cross. We then formalise a set of properties that verify its correctness and check them in mCRL2.

Processes are written in the CCS-style notation used in the course, and properties in process logic with regular expressions, where $-$ denotes any action, $-a$ any action other than a, and ; sequential composition (respectively `true`, `!a` and `.` in mCRL2).

Task 1: One train

1a. Components

$$\begin{aligned} \text{Train} &= \text{approach} \,.\, \text{grant} \,.\, \text{enter} \,.\, \text{exit} \,.\, \text{leave} \,.\, \text{Train} \\ \text{Gate} &= \text{lower} \,.\, \text{down} \,.\, \text{raise} \,.\, \text{up} \,.\, \text{Gate} \\ \text{Ctrl} &= \text{approach} \,.\, \text{lower} \,.\, \text{down} \,.\, \text{grant} \,.\, \text{leave} \,.\, \text{raise} \,.\, \text{up} \,.\, \text{Ctrl} \end{aligned}$$

Each component is a single recursive loop, so all ordering constraints live in the controller: the grant is only issued once the gate reports `down`, and the gate is only raised after the train has signalled `leave`.

1b. Composition

Every interaction is modelled as a pair of complementary actions (e.g. `approach_train` and `approach_controller`) that `comm` merges into one synchronised action. `allow` then keeps only the synchronised results, so neither half can occur on its own:

```mcrl2
hide({ lower, down, grant, leave, raise, up },
allow({ approach, lower, down, grant, leave, raise, up, enter, exit },
comm({ approach_controller | approach_train -> approach, ... },
      TRAIN || GATE || CONTROLLER )));
```

For inspection we hide the internal handshakes with the gate, as well as `grant` and `leave`. We deliberately keep `approach` visible alongside `enter` and `exit`, so the observable behaviour still records a train arriving, entering and exiting.

1c. Inspection

TODO: screenshot 1_lts.png

Figure 1: Full LTS

TODO: screenshot 1_lts_reduced.png

Figure 2: Reduced LTS

TODO: describe the full LTS and what remains in the reduced one.

1d. Properties

Property	Formula	Result
Safety	$[-^*; \text{enter}; (-\text{exit})^*; \text{up}; -^*; \text{exit}]\ \text{false} \wedge$ $[-^*; \text{up}; (-\text{down})^*; \text{enter}]\ \text{false}$	true
Reachability	$[-^*; \text{leave}]\langle -^*; \text{raise}\rangle\ \text{true}$	true
Liveness	$[-^*; \text{approach}]\langle -^*; \text{exit}\rangle\ \text{true}$	true
Deadlock freedom	$[-^*]\langle -^*; \text{approach}\rangle\ \text{true}$	true

Safety has two parts: the gate never goes `up` while the train is between `enter` and `exit`, and after an `up` the train never enters before a `down`. With a single train every path follows the same cycle, so "can exit" in Liveness is the same as "must exit".

1e. Break it

TODO: the faulty variant, which property fails, the counterexample trace, and what went wrong.

Task 2: Multiple trains

2a. Model with data

We chose the **queue model**. The controller stores a list Q of the trains that approached and did not leave yet, in arrival order. It also stores `crossing` (c, `busy` from a `grant` until the matching `leave`) and its own view of the gate (g). The gate is the same as in Task 1. As in Task 1, the controller only talks to a train through `approach`, `grant` and `leave`: `enter` and `exit` are the train's own observable actions.

$$\text{Train}(t) = \text{approach}(t) \,.\, \text{grant}(t) \,.\, \text{enter}(t) \,.\, \text{exit}(t) \,.\, \text{leave}(t) \,.\, \text{Train}(t)$$

$$\text{Gate} = \text{lower} \,.\, \text{down} \,.\, \text{raise} \,.\, \text{up} \,.\, \text{Gate}$$

$$\begin{aligned}\text{Ctrl}(Q, c, g) = \sum_{t \in \text{Id}} (t \notin Q) \to \text{approach}(t) \,.\, \text{Ctrl}(Q \triangleleft t, c, g)\\ +(Q \neq [\,] \wedge g = \text{closed} \wedge c = \text{free}) \to \text{grant}(\text{head}(Q)) \,.\, \text{Ctrl}(Q, \text{busy}, g)\\ +(Q \neq [\,] \wedge c = \text{busy}) \to \text{leave}(\text{head}(Q)) \,.\, \text{Ctrl}(\text{tail}(Q), \text{free}, g)\\ +(g = \text{open} \wedge Q \neq [\,]) \to \text{lower} \,.\, \text{Ctrl}(Q, c, \text{closing})\\ +(g = \text{closing}) \to \text{down} \,.\, \text{Ctrl}(Q, c, \text{closed})\\ +(g = \text{closed} \wedge Q = [\,] \wedge c = \text{free}) \to \text{raise} \,.\, \text{Ctrl}(Q, c, \text{opening})\\ +(g = \text{opening}) \to \text{up} \,.\, \text{Ctrl}(Q, c, \text{open})\end{aligned}$$

$$\text{System} = \nabla_V(\Gamma_C(\text{Train}(t_1) \parallel \ldots \parallel \text{Train}(t_N) \parallel \text{Gate} \parallel \text{Ctrl}([\,], \text{free}, \text{open})))$$

where $Q \triangleleft t$ appends t at the end of Q (`<|` in mCRL2).

Why a queue. With a set, `grant` could pick any waiting train, so one train could wait forever while another kept crossing. With a queue only the head can be granted, and a train that crosses and approaches again goes to the back. So no train is ever overtaken.

Why the gate always finishes opening. Once the controller decides to `raise`, the gate completes `up`, even if a train approached in the meantime. The controller then sees a non-empty queue and lowers it again. The train is not affected: it still needs `lower`, `down` and `grant` before it can enter.

Invariant. Q always holds exactly the trains that did `approach` but not yet `leave`, in arrival order and without repetitions. It starts empty, `approach(t)` is the only step that adds t (at the end), and `leave` the only one that removes a train (the head). The train that leaves is always the head, because only the head can be granted and nothing gets in front of it. Likewise, $c = \text{busy}$ holds exactly while one train is between `grant` and `leave`, because `grant` needs $c = \text{free}$. Two consequences follow:

- at most one train is between `grant` and `leave` at any time;
- `raise` needs $Q = [\,]$ and $c = \text{free}$, so the controller only decides to open when no train is between `approach` and `leave`.

2b. Properties

All for every $t \in \text{Id}$ (and $t \neq t'$ in 2 and 5b):

Property	Formula	Result
1. Safety	$[-^*; \text{enter}(t); (-\text{leave}(t))^*; \text{up}]\ \text{false}$	true
2. Mutual exclusion	$[-^*; \text{enter}(t); (-\text{exit}(t))^*; \text{enter}(t')]\ \text{false}$	true
3. No premature raise	$[-^*; \text{approach}(t); (-\text{leave}(t))^*; \text{raise}]\ \text{false}$	true
4. Deadlock freedom	$[-^*]\langle -^*; \text{approach}(t)\rangle\ \text{true}$	true
5a. Liveness (reach)	$[-^*; \text{approach}(t); (-\text{enter}(t))^*]\langle -^*; \text{enter}(t)\rangle\ \text{true}$	true
5b. Liveness (order)	$[-^*; \text{approach}(t); (-\text{enter}(t))^*; \text{enter}(t');$ $(-\text{enter}(t))^*; \text{enter}(t')]\ \text{false}$	true
6. Road availability	$[-^*]\langle -^*; \text{up}\rangle\ \text{true}$	true
7. Infinite opening	$[((-\text{up})^*; \text{up})^*]\ \text{true} \wedge \langle -\rangle\ \text{true}$	true
8. Non-vacuity	$[-^*]\langle -^*; \text{enter}(t)\rangle\ \text{true} \wedge [-^*]\langle -^*; \text{raise}\rangle\ \text{true} \wedge [-^*]\langle -\rangle\ \text{true}$	true

Properties 1–3 express the invariant directly. Property 3 checks `raise`, the controller's decision to open, as in the assignment ("the gate is never raised"); an `up` may still finish an opening started before the train arrived. Since a box formula holds trivially when its premise is unreachable, property 8 checks that the premises of 1–3 are in fact reachable, and that there is always a next step.

Liveness and fairness (5). Without fixpoints we cannot write "every `approach(t)` **must** be followed by `enter(t)`", so property 5 has two parts. The first (5a) says that while t waits, `enter(t)` stays reachable. The second (5b) says that while t waits, no other train enters twice, i.e. t is overtaken at most $N - 1$ times. It is restricted to $t' \neq t$, because for $t' = t$ the pattern matches t's own next trip ($\text{enter}(t)$, then later $\text{enter}(t)$ again), which is normal behaviour. With a set the second part fails, and we would need to assume fairness between trains. With the queue it holds, so a waiting train can only be delayed by the trains already ahead of it, and those are finite. We therefore need no fairness assumption for liveness, only that the system keeps running, which property 4 shows. The step from these two parts to "must enter" is our argument, not something the tool checks.

Infinite opening (7). Fairness towards the road cannot come from the controller. If trains keep arriving so that Q is never empty, `raise` never happens and the gate stays closed. In practice, cars wait for as long as the train traffic lasts. Avoiding this would mean holding trains back to let cars through, which changes what the system is for. So this property depends on an assumption about the trains: that there are moments when none is approaching. Property 6 shows that opening is always still possible.

2c. Break it: a family of bugs

TODO: naive raise, missed increment, off-by-one. For each: the change, which properties fail, the shortest counterexample and the scenario in words.

2d. State space

N	States	Transitions	States (red.)	Trans. (red.)
1	TODO	TODO	TODO	TODO
2	TODO	TODO	TODO	TODO
3	TODO	TODO	TODO	TODO

4	TODO	TODO	TODO	TODO
5	TODO	TODO	TODO	TODO
6	TODO	TODO	TODO	TODO

The state space grows quickly with N: each train moves through its own cycle independently, and the controller stores not just which trains are waiting but also their order. We could still generate it for $N = 6$. The LTS has a star shape in which $N - 1$ arms always have the same shape. This reflects the symmetry between trains: they only differ by their identifier, so permuting identifiers maps the system onto itself.

Conclusion

With a single train, the fixed order of actions is enough to guarantee safety. With N trains, the controller must track which trains are near (Q) and whether the crossing is occupied (c), and these two invariants are what make safety, mutual exclusion and no premature raising hold. Keeping Q as a queue also gives liveness per train without a fairness assumption. Fairness only remains an issue for the road, which stays closed for as long as trains keep arriving.